

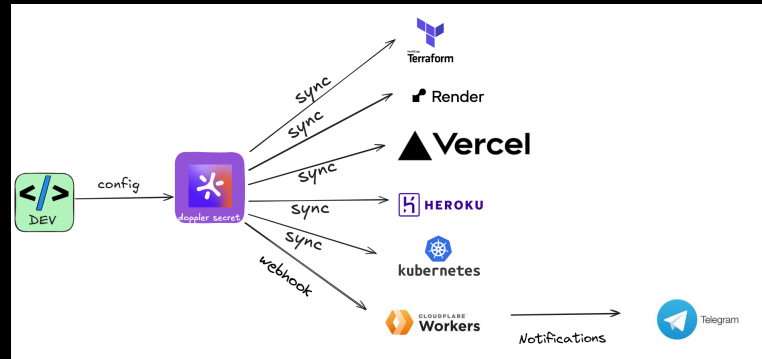
MỤC LỤC

0.1	Kiến trúc hệ thống	2
0.1.1	Quản lý các thông tin bí mật (doppler)	2
0.1.2	CI/CD (Github Action)	2
0.1.3	Cơ sở hạ tầng dưới dạng mã (Infrastructure as Code - Terraform)	3
0.1.4	API Gateway trong Microservice (Kong API Gateway)	6
0.1.5	Container và GitOps (Docker, Kubernetes, ArgoCD)	11

0.1 Kiến trúc hệ thống

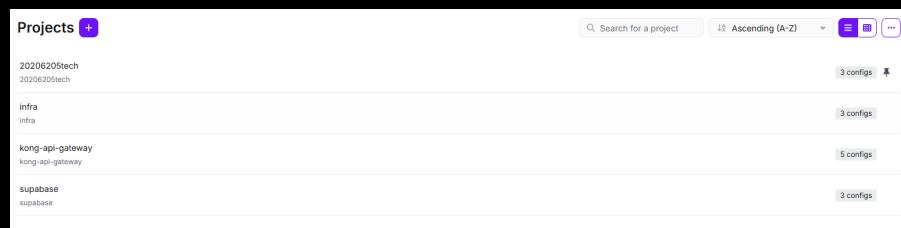
0.1.1 Quản lý các thông tin bí mật (doppler)

Quản lý các thông tin bí mật là một phần cực kỳ quan trọng. Đặc biệt khi hệ thống được thiết kế theo kiến trúc microservices. Khi dự án có nhiều service, việc sử dụng file .env độc lập cho từng service sẽ gây khó khăn trong việc quản lý, cấu hình, đồng bộ và bảo mật.



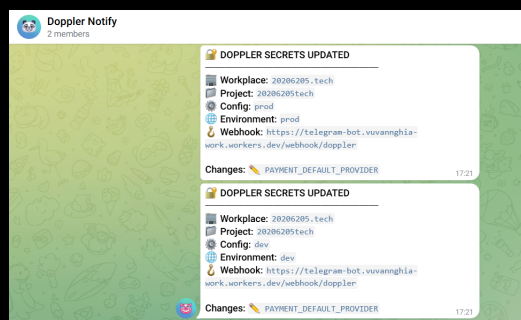
Hình 0.1: Sơ đồ quản lý secret bằng doppler

Trong đồ án này, em sử dụng Doppler quản lý thông tin bí mật tập trung. Việc áp dụng Doppler mang lại một không gian lưu trữ an toàn, thống nhất (Single Source of Truth) cho toàn bộ hệ thống. Các bí mật được chia nhỏ theo từng nhu cầu sử dụng trong doppler.



Hình 0.2: Cấu hình chia nhỏ bí mật theo từng nhu cầu sử dụng trong doppler

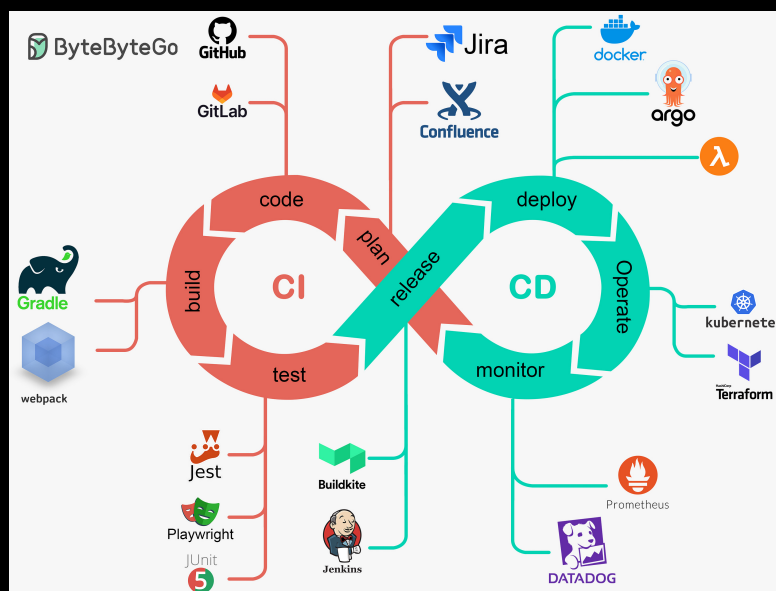
Trong hệ thống Doppler, bản miễn phí có hỗ trợ nhánh dev personal riêng cho từng lập trình viên, từng cá nhân có thể ghi đè hoặc thêm các biến mà không làm ảnh hưởng đến cấu hình chung. Trong bản trả phí, Doppler hỗ trợ tính năng xoay vòng tự động các biến bí mật.



Hình 0.3: Kết quả thông báo telegram khi doppler có sự thay đổi

0.1.2 CI/CD (Github Action)

CI/CD là phương pháp tự động hóa quy trình tích hợp và triển khai phần mềm. Phương pháp này giúp đẩy nhanh tốc độ, giảm thiểu lỗi thủ công và duy trì độ ổn định cho hệ thống. GitHub Actions là công cụ CI/CD mạnh mẽ được tích hợp sẵn trên GitHub.

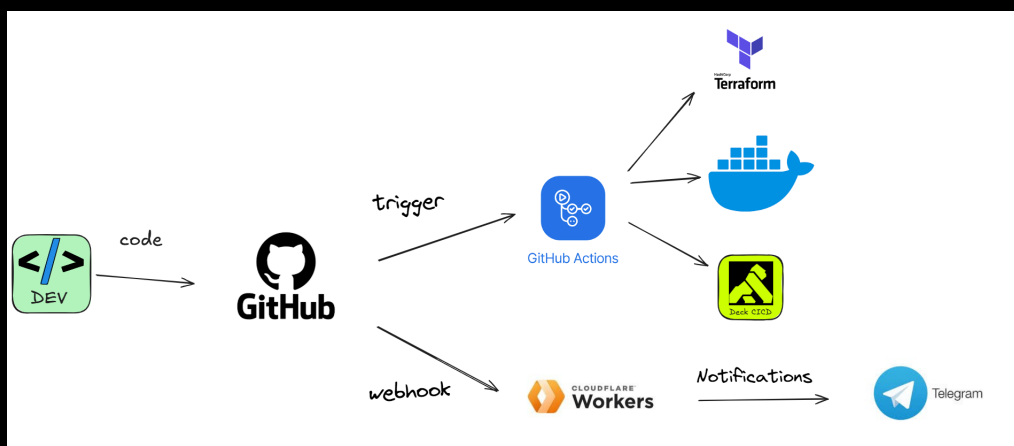


Hình 0.4: Ví dụ tổng quan CI/CD

Ứng dụng trong đồ án

Trong đồ án này, hệ thống vừa sử dụng Github Action miễn phí của Github, vừa sử dụng Self-hosted runner trên VPS.

Sơ đồ tổng quan CI/CD bằng Github Action trong dự án



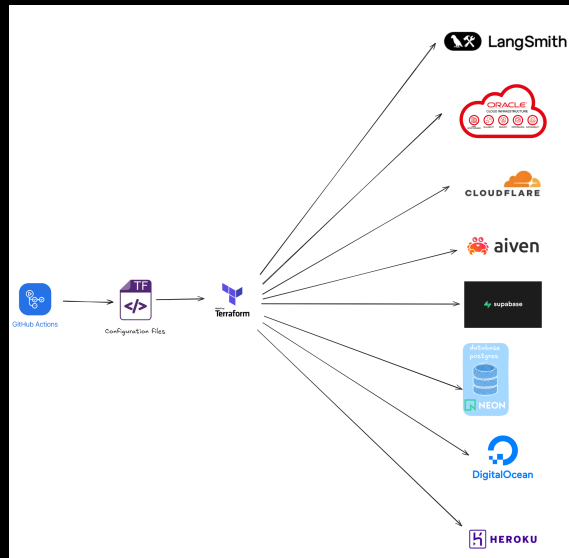
Hình 0.5: Sơ đồ tổng quan CI/CD bằng Github Action trong dự án

0.1.3 Cơ sở hạ tầng dưới dạng mã (Infrastructure as Code - Terraform)

Trong kỹ thuật phần mềm, việc cấu hình cơ sở hạ tầng theo phương pháp thủ công thường tiềm ẩn nhiều rủi ro gây sai sót, tốn thời gian và gặp khó khăn trong việc tái khởi tạo một cách chính xác. Ngoài ra, việc dọn dẹp hoặc hủy bỏ tài nguyên thủ công dễ dẫn đến tình trạng sót tài nguyên, gây lãng phí và rủi ro bảo mật. Việc sử dụng Infrastructure as Code (IaC) sẽ khắc phục được vấn đề đó.

Ứng dụng trong đồ án

Trong phạm vi đồ án này, em sử dụng Terraform để quản lý cấu hình một số hạ tầng của các nhà cung cấp có hỗ trợ terraform giúp định nghĩa và quản lý hạ tầng của dự án một cách tự động thay vì thao tác thủ công.



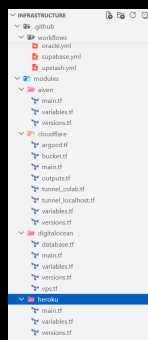
Hình 0.6: Sơ đồ tổng quan quản lý hạ tầng bằng Terraform trong dự án

Hạ tầng của hệ thống quản lý qua các dịch vụ bao gồm:

- **Oracle Cloud Infrastructure (OCI):** Khởi tạo và quản lý máy chủ ảo (VPS) phục vụ triển khai ứng dụng.
- **Aiven:** Khởi tạo Kafka trung chuyển tin nhắn hệ thống.
- **Supabase:** Cấu hình hệ thống và các chính sách cho dịch vụ xác thực người dùng (Auth Service).
- **LangSmith:** Quản lý các tài nguyên phục vụ giám sát và đánh giá chuỗi RAG (LangChain, LangGraph).
- **Cloudflare:** Quản lý cấu hình DNS, lưu trữ đối tượng (Cloudflare R2 Bucket) và thiết lập các đường truyền an toàn (Cloudflare Tunnel).
- **Neon:** Khởi tạo và quản lý các CSDL PostgreSQL độc lập cho từng vi dịch vụ (Microservices).
- **DigitalOcean:** Cấu hình CSDL PostgreSQL phục vụ riêng cho luồng thu thập dữ liệu (Data Pipeline VBPL).
- **Heroku:** Triển khai và quản lý môi trường cho dịch vụ thanh toán (Payment Service).

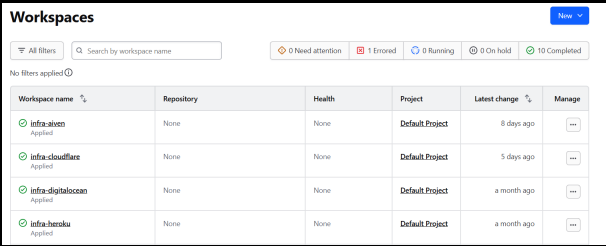
Tại dự án infra-by-terraform dùng root module lớn truyền biến vào các module nằm trong thư mục modules. Tuy nhiên việc chạy CI/CD có thể xảy ra lỗi vì provider bị lỗi, hoặc mất thời gian khi hệ thống bắt buộc phải thực hiện lệnh `terraform init` cho toàn bộ các *provider* mặc dù chỉ có một thay đổi rất nhỏ ở một dịch vụ cụ thể.

Vì vậy, trong đồ án này, em sử dụng mỗi provider trong folder riêng tại dự án infrastructure. Kết hợp github action chỉ kích hoạt quy trình kiểm tra và triển khai Terraform khi và chỉ khi có sự thay đổi dữ liệu bên trong thư mục của *provider* tương ứng.



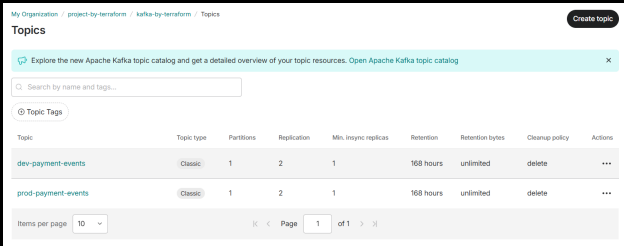
Hình 0.7: Cấu trúc thư mục dự án Terraform theo từng Provider

Bên cạnh đó, dự án sử dụng giải pháp Terraform Cloud đóng vai trò làm *Remote Backend* để lưu trữ trạng thái (*State File*).

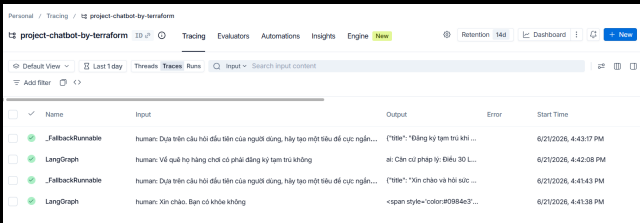


Hình 0.8: Giao diện quản lý trạng thái tập trung trên Terraform Cloud

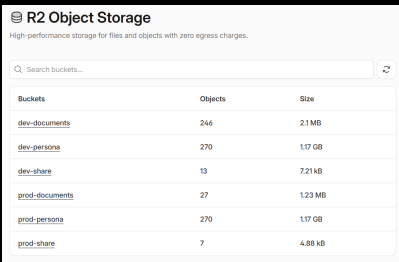
Dưới đây là một vài kết quả các tài nguyên được khởi tạo tự động thông qua mã nguồn Terraform thành công:



Hình 0.9: Dịch vụ Kafka trên nền tảng Aiven được cấu hình tự động

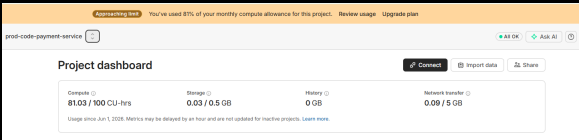


Hình 0.10: Dự án LangSmith được tạo từ terraform cho dịch vụ chatbot



Hình 0.11: Cấu hình các R2 Bucket thành công trên Cloudflare

Do chính sách tài khoản miễn phí của Neon có nhiều giới hạn về tài nguyên, trong mã nguồn Terraform, em đã cấu hình tách biệt hoàn toàn thành các dự án riêng lẻ trên Neon cho từng môi trường và dịch vụ. Từ đó, đảm bảo sự độc lập dữ liệu và tránh giới hạn.



Hình 0.12: Giao diện giám sát giới hạn tài nguyên của tài khoản miễn phí trên Neon

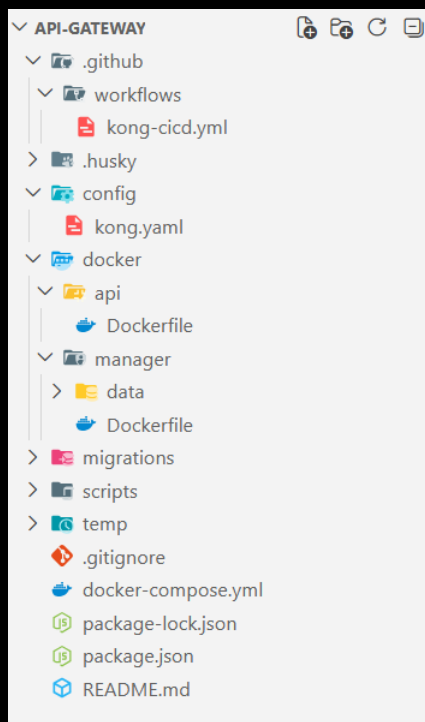
0.1.4 API Gateway trong Microservice (Kong API Gateway)

API Gateway được sử dụng để quản lý và kiểm soát việc truy cập vào các API. API Gateway cung cấp một lớp trung gian giữa client và các dịch vụ. Có thể nói, API Gateway là một điểm vào độc lập cho nhiều client của một ứng dụng. Kong API Gateway là một trong những API Gateway mã nguồn mở phổ biến có thể quản lý các API được triển khai ở bất kỳ đâu, từ cơ sở hạ tầng đơn giản đến môi trường đa đám mây phức tạp.



Hình 0.13: Sơ đồ tổng quan kiến trúc và luồng hoạt động của Kong API Gateway

Cấu trúc dự án Kong API Gateway



Hình 0.14: Cấu trúc dự án Kong API Gateway

Trong môi trường sản xuất thực tế gặp nhiều thách thức, khó khăn. Em sử dụng các plugin của Kong API Gateway như rate limit giới hạn tốc độ yêu cầu, bot-detection ngăn chặn các yêu cầu độc hại.

id	uri	method	status	createdAt
00028917-acfd-46e3-894...	/config/mailgun_keys.php	GET	404	2026-05-15 14:45:33.76...
00103ebf-7f3c-4f1b-b7b...	/config/sendinblue.yml	GET	404	2026-05-16 06:45:10.24...
001228a9-cfba-40a7-84c...	//edit.php	GET	404	2026-05-23 07:26:23.59...
001416ff-6015-4855-bbb...	/sendgrid.json	GET	404	2026-05-16 06:45:38.89...
0014726f-525d-46d4-98d...	/config/email.php~	GET	404	2026-05-16 06:49:20.29...
00159dac-2cc0-4f16-a7f...	/.aws/credentials	GET	404	2026-05-16 06:38:52.73...
00171fd3-0ea0-47fe-b70...	/wp-content/plugins/hellopress/wp_filemanager.php	GET	404	2026-06-12 02:35:19.16...
001a8ccb-fdff-4e8a-91f...	/config/mandrill.staging.php	GET	404	2026-05-16 06:46:12.72...
001de6eb-abal-47a7-8a3...	/	GET	404	2026-05-08 17:02:25.01...
001ef7d0-d720-461c-955...	/backups/config/mail.php	GET	404	2026-05-16 06:50:12.62...
00270a2b-3065-4529-ae1...	/application/email.php	GET	404	2026-05-16 06:49:56.30...
002d344f-597e-42f7-bfa...	/dx.php	GET	404	2026-05-09 14:49:52.41...
002e13b8-9714-4bfe-a7a...	/dev/.env	GET	404	2026-05-20 13:10:27.05...
0030f201-b836-4a6e-b9c...	/cd/.env	GET	404	2026-05-08 20:11:31.81...
003b1ccc-32be-4002-ba6...	/phpinfo	GET	404	2026-05-12 22:00:06.41...
003b91e6-7cb2-4924-87a...	/react/.env	GET	404	2026-05-20 13:10:38.03...
003dc46f-5358-46c9-b8f...	/config/services/email.php	GET	404	2026-05-16 06:41:00.74...
0042f492-a7c8-489e-8a2...	/beta/.env	GET	404	2026-05-20 13:10:52.29...
0043577c-4f28-4a1d-8e4...	/wp-content/plugins/mailjet-smtp/mailjet-smtp.php	GET	404	2026-05-16 06:43:31.08...

Hình 0.15: Các yêu cầu độc hại dò các bí mật nhạy cảm (404 Not Found)



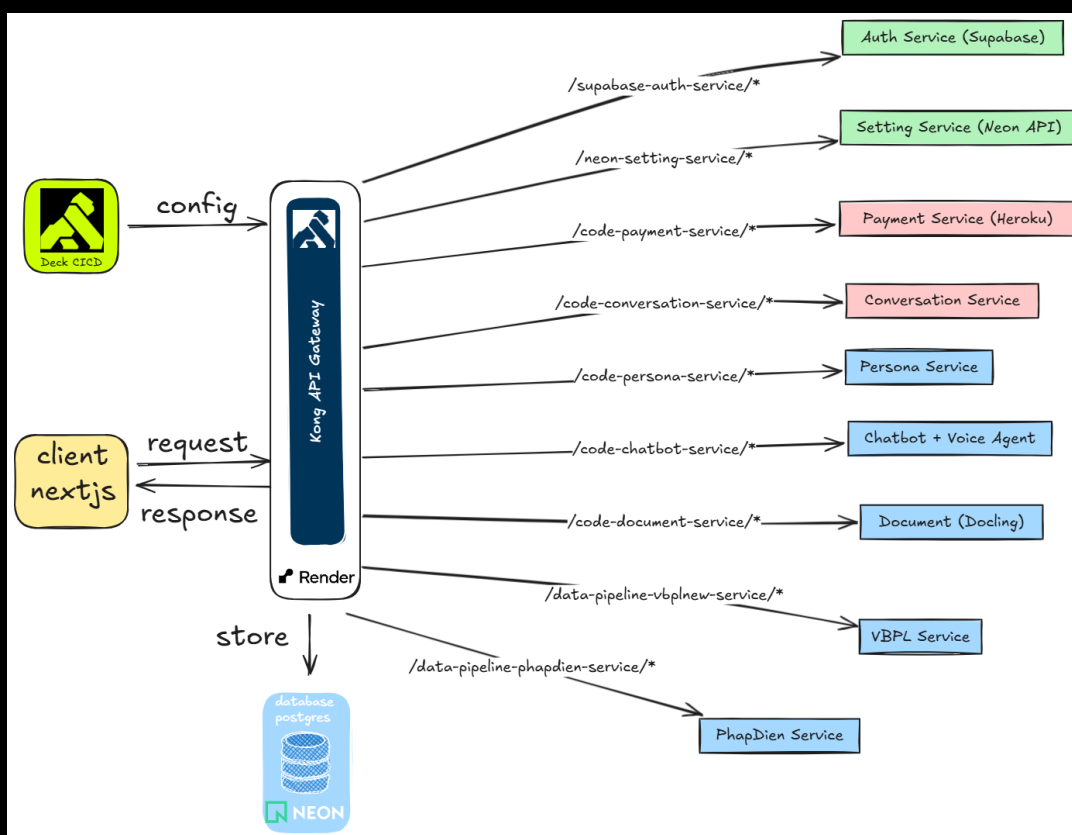
Hình 0.16: Thiết lập định tuyến dịch vụ trong Kong API Gateway

Kong API Gateway có hỗ trợ nhiều cách quản lý cấu hình khác nhau như Kong Admin API (REST API), Terraform hay qua các giao diện UI. Tuy nhiên, Deck lại là công cụ được chính Kong phát triển để tối ưu hóa riêng cho việc quản lý cấu hình. Với Terraform, nếu ai đó vào giao diện hoặc gọi REST API sửa đổi một cấu hình của Kong, file terraform.tfstate sẽ bị lệch, dẫn đến việc apply tiếp theo có thể lỗi hoặc dè dặt dữ liệu. Với Deck sẽ đọc trực tiếp cấu hình hiện tại trong database của Kong, so sánh (diff) với file YAML và chỉ cập nhật những phần thay đổi qua API dạng batch tối ưu.

Không sợ bị lệch trạng thái. Ngoài ra, Deck hỗ trợ CI/CD với GitHub Action (kong/setup-deck). Lưu trữ toàn bộ cấu hình Kong API Gateway với file kong.yaml vào một kho chứa Git Thiết lập các pipeline để tự động kiểm tra cú pháp deck gateway validate kong.yaml, tự động so sánh thay đổi deck gateway diff kong.yaml và tự động cập nhật deck gateway sync kong.yaml khi có Pull Request được merge.



Hình 0.17: Luồng CI/CD triển khai cấu hình Kong API Gateway qua GitHub Actions



Hình 0.18: Sử dụng Deck và database postgres cấu hình Kong API Gateway

Các plugins của Kong API Gateway trong dự án này:

- Plugin bot-detection: phân tích User-Agent và các dấu hiệu khác để chặn các request từ các

bot độc hại, scraper, hoặc các công cụ tự động không mong muốn.

- Plugin rate-limiting: giới hạn số lượng request, giúp bảo vệ các dịch vụ khỏi bị quá tải hoặc bị lạm dụng.
- Plugin request-size-limiting: chặn các request có kích thước vượt quá ngưỡng cho phép.
- Plugin http-log: thu thập các thông tin quan trọng về request/response và gửi qua giao thức http api NestJS và thực hiện lưu vào database postgres
- Plugin correlation-id tạo request-id: tự động tạo ra một mã định danh duy nhất và gắn vào header của mỗi request để truy vết (tracing).
- Plugin request-transformer thêm thông tin Request-Kong-Secret: gửi header chứa mã bí mật vào request trước khi đẩy xuống dịch vụ để các request phải được gọi từ Kong API Gateway.

Name	Applied To	Status
 Request Transformer	Global	Enabled ☒
 Request Transformer	Route	Enabled ☒
 Request Termination	Route	Enabled ☒
 Request Transformer	Service	Enabled ☒
 Correlation ID	Global	Enabled ☒
 Request Size Limiting	Global	Enabled ☒
 Bot Detection	Global	Enabled ☒
 Rate Limiting	Global	Enabled ☒
 HTTP Log	Global	Enabled ☒
 Request Transformer	Route	Enabled ☒
 Request Transformer	Service	Enabled ☒

Hình 0.19: Danh sách các plugin được cấu hình trên Kong API Gateway

Các kết quả phản hồi Kong API Gateway:

Curl

```
curl -X 'GET' \
  'http://134.185.92.211/code-chatbot-service/chats?skip=0&limit=100' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer example'
```

Request URL

```
http://134.185.92.211/code-chatbot-service/chats?skip=0&limit=100
```

Server response

Code	Details
400 <i>Undocumented</i>	Error: Bad Request Response body <pre>{ "detail": "Request-Id header is required" }</pre>

Hình 0.20: Phản hồi khi yêu cầu thiếu mã định danh Request ID

Curl

```
curl -X 'GET' \
  'http://134.185.92.211/code-chatbot-service/chats?skip=0&limit=100' \
  -H 'accept: application/json' \
  -H 'request-id: df82eb45-8517-4578-81ca-58f078bdfb9c' \
  -H 'Authorization: Bearer example'
```

Request URL

```
http://134.185.92.211/code-chatbot-service/chats?skip=0&limit=100
```

Server response

Code	Details
400 <i>Undocumented</i>	Error: Bad Request Response body <pre>{ "detail": "Request-Kong-Secret header is required" }</pre>

Hình 0.21: Phản hồi khi yêu cầu thiếu mã bảo mật Kong Secret

Curl

```
curl -X 'GET' \
  'http://134.185.92.211/code-chatbot-service/chats?skip=0&limit=100' \
  -H 'accept: application/json' \
  -H 'request-id: df82eb45-8517-4578-81ca-58f078bdfb9c' \
  -H 'request-kong-secret: example' \
  -H 'Authorization: Bearer example'
```

Request URL

```
http://134.185.92.211/code-chatbot-service/chats?skip=0&limit=100
```

Server response

Code	Details
401 <i>Undocumented</i>	Error: Unauthorized Response body <pre>{ "detail": "Invalid Request-Kong-Secret" }</pre>

Hình 0.22: Phản hồi khi mã bảo mật Kong Secret không hợp lệ

0.1.5 Container và GitOps (Docker, Kubernetes, ArgoCD)

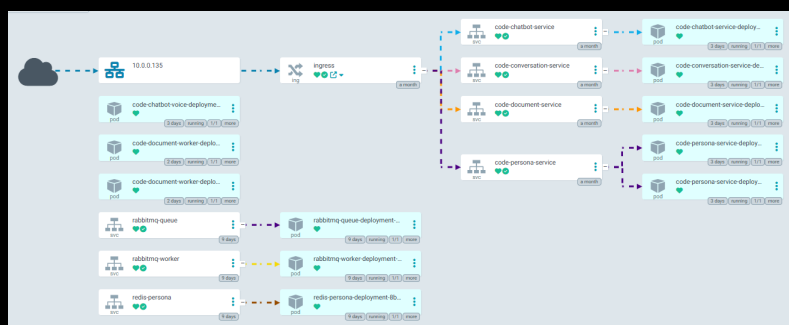
- **Docker:** Hệ thống chạy tốt trên máy của lập trình viên nhưng lỗi khi lên server do khác biệt về hệ điều hành hoặc thư viện. Docker đóng gói ứng dụng cùng với các thư viện, file cấu hình và môi trường cần thiết vào một hộp kín gọi là Container.
- **Kubernetes (K8s):** Khi có nhiều container trên các server khác nhau, nếu một server down thì sao? Nếu đột nhiên lượng truy cập tăng vọt thì làm thế nào? Kubernetes là một hệ thống công cụ quản lý container (container orchestration). Giúp tự động hóa việc triển khai, mở rộng quy mô và quản lý các container. Kubernetes có khả năng thực hiện tính năng Rolling Update. Đây là một trong những cơ chế đảm bảo không gián đoạn dịch vụ (Zero Downtime) bằng cách tạo từ từ các Pod mới và lần lượt tắt các Pod cũ. Kubernetes không xóa tất cả các Pod cùng một lúc. Điều này đảm bảo ứng dụng của bạn luôn có Pod đang chạy để xử lý request từ người dùng.
- **GitOps với ArgoCD:** Việc cấu hình hệ thống bằng cách gõ lệnh tay rất dễ dẫn đến sai sót, khó theo dõi ai đã thay đổi gì và cực kỳ khó khôi phục nếu hệ thống sập. GitOps là một phương pháp sử dụng manifest của Git (như GitHub, GitLab) là nguồn chân lý duy nhất (Single Source of Truth). Các file yaml của K8s đều được lưu trên Git. ArgoCD nhận diện thay đổi và tự động triển khai (deploy) hoặc tinh chỉnh hạ tầng thực tế cho khớp với Git.

Docker image code-chatbot-service được đặt là private vì image này quan trọng nhất của toàn bộ dự án. Kết quả các Docker Image được push lên Docker Hub:

Repositories				
All repositories within the vuvannghiawork namespace.				
Search by repository name All content		Create a repository		
Name	Last Pushed	Contains	Visibility	Scout
vuvannghiawork/code-conversation-service	about 3 hours ago	BUILDKIT_CACHE IMAGE	Public	Inactive
vuvannghiawork/code-document-worker	about 7 hours ago	IMAGE	Public	Inactive
vuvannghiawork/code-chatbot-service	about 13 hours ago	BUILDKIT_CACHE IMAGE	Private	Inactive
vuvannghiawork/code-persona-service	about 13 hours ago	IMAGE	Public	Inactive
vuvannghiawork/code-document-service	1 day ago	BUILDKIT_CACHE IMAGE	Public	Inactive
vuvannghiawork/data-pipeline-vbpl-service	3 days ago	BUILDKIT_CACHE IMAGE	Public	Inactive

Hình 0.23: Kết quả các Docker Image được push lên Docker Hub

Kết quả giao diện ArgoCD chạy triển khai các docker iamge trong kubernetes:



Hình 0.24: Kết quả giao diện ArgoCD chạy docker trong kubernetes

Các quyết định cấu hình Kubernetes trong kho lưu trữ GitOps của dự án này:

- Triển khai các dịch vụ trung gian trong cụm (RabbitMQ và Redis):** Triển khai RabbitMQ và Redis cùng các dịch vụ khác trực tiếp trong cụm Kubernetes giúp tăng tốc độ xử lý, loại bỏ độ trễ mạng (network latency), không bị giới hạn tài nguyên như các gói miễn phí (ví dụ: cloudAMQP) và không cần gọi kết nối ra bên ngoài (redis.io) như dịch vụ thanh toán được triển khai ở Heroku.
- Cấu hình loại dịch vụ là ClusterIP cho các hàng đợi và bộ nhớ đệm:** Cấu hình RabbitMQ queue (kết nối conversation service và chatbot service), RabbitMQ worker (kết nối document service và document worker) và Redis (lưu trữ cache của persona service) có loại dịch vụ (type) là ClusterIP. Thiết lập này giúp ngăn chặn hoàn toàn các client từ bên ngoài cụm Kubernetes kết nối trực tiếp đến các dịch vụ nội bộ này.
- Triển khai dịch vụ giọng nói (Voice Deployment - LiveKit Agent):** Dịch vụ giọng nói không cấu hình service đi kèm. Do đó, các tài nguyên khác (cả bên trong và bên ngoài cụm Kubernetes) không thể chủ động định tuyến và gửi các request trực tiếp vào các Pod của voice agent. Thay vào đó, voice agent hoạt động như một LiveKit Agent chủ động lắng nghe và kết nối trực tiếp với máy chủ LiveKit.
- Triển khai dịch vụ xử lý tài liệu (Document Worker - Celery):** Tương tự như voice agent, dịch vụ document worker (sử dụng Celery) cũng không cấu hình service đi kèm. Document worker hoạt động theo cơ chế chủ động lắng nghe hàng đợi từ RabbitMQ worker để xử lý các tác vụ bất đồng bộ.
- Cấu hình gRPC:** Dịch vụ gRPC chạy trên cổng PORT 50051 nhưng được cấu hình với loại dịch vụ là ClusterIP và không cấu hình Ingress. Thiết lập này ngăn chặn các client bên ngoài cụm Kubernetes kết nối trực tiếp đến gRPC. Nhờ đó, mọi yêu cầu từ client bắt buộc phải đi qua conversation service để kiểm tra nghiệp vụ gói thanh toán VIP trước khi gửi đến chatbot service.

Chữ tạm màu đỏ
Đồng thứ hai vẫn có màu đỏ và không bị lỗi ngắt đoạn.
<https://example.com>



Hình 0.25: Logo của Đại học Bách Khoa Hà Nội



Hình 0.26: Logo của Đại học Bách Khoa Hà Nội



Hình 0.27: Logo của Đại học Bách Khoa Hà Nội

- Nội dung mục 1
- Chấm màu xanh

In nghiêng đoạn văn bản

In đậm đoạn văn bản

Ứng dụng trong đồ án

Nội dung block

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.